

criptografia-simetrica-cesar



Criptografia Simétrica — Cifra de César



Atividade de Segurança da Informação sobre **Cifra de César**, com cifração manual, decifração por força bruta e observações da implementação.



Tarefa 1 — Cifração Manual



Objetivo: cifrar a mensagem usando **k = 7**, mantendo os espaços.

1) Alfabeto (A–Z) com posições

- A=0, B=1, C=2, D=3, E=4, F=5, G=6, H=7, I=8, J=9, K=10, L=11, M=12, N=13, O=14, P=15, Q=16, R=17, S=18, T=19, U=20, V=21, W=22, X=23, Y=24, Z=25

2) Fórmula

- Cifração:** $C = (P + k) \bmod 26$
- Com **k = 7**: $C = (P + 7) \bmod 26$

3) Aplicação letra por letra

Palavra 1: SEGURANCA

- S** (18) + 7 = 25 → **Z**
- E** (4) + 7 = 11 → **L**
- G** (6) + 7 = 13 → **N**
- U** (20) + 7 = 27 → $27 \bmod 26 = 1$ → **B**

- **R** (17) + 7 = 24 → **Y**
- **A** (0) + 7 = 7 → **H**
- **N** (13) + 7 = 20 → **U**
- **C** (2) + 7 = 9 → **J**
- **A** (0) + 7 = 7 → **H**
- **Resultado parcial: ZLNBYHUJH**

Palavra 2: DA

- **D** (3) + 7 = 10 → **K**
- **A** (0) + 7 = 7 → **H**
- **Resultado parcial: KH**

Palavra 3: INFORMACAO

- **I** (8) + 7 = 15 → **P**
- **N** (13) + 7 = 20 → **U**
- **F** (5) + 7 = 12 → **M**
- **O** (14) + 7 = 21 → **V**
- **R** (17) + 7 = 24 → **Y**
- **M** (12) + 7 = 19 → **T**
- **A** (0) + 7 = 7 → **H**
- **C** (2) + 7 = 9 → **J**
- **A** (0) + 7 = 7 → **H**
- **O** (14) + 7 = 21 → **V**

Texto cifrado resultante

| ZLNBYHUJH KH PUMVYTHJHV

Evidências

Antes

Atividade Prática — Parte 1: Cifração Manual

Tarefa 1 — Cifre a mensagem

Usando a **Cifra de Cesar** com $k = 7$, cifre a seguinte mensagem:

SEGURANCA DA INFORMACAO

Passos:

- 1 Escreva o alfabeto de A–Z com suas posicoes (A=0, B=1, ...);
- 2 Para cada letra da mensagem, some 7 e aplique modulo 26;
- 3 Anote o texto cifrado resultante.

Depois

```
PS E:\01-Geral\PUC\9 Período\Segurança_d_inf\Entregas\criptografia-simetrica-cesar\s
rc> javac CifraCesar.java
PS E:\01-Geral\PUC\9 Período\Segurança_d_inf\Entregas\criptografia-simetrica-cesar\s
rc> java CifraCesar
Caso queira decifrar, basta mandar k negativo.
Digite a mensagem a ser cifrada: SEGURANCA DA INFORMACAO
Digite o valor da chave (k): 7

--- PROCESSANDO ---
Texto Cifrado   : ZLNBYHUJH KH PUMVYTHJHV
Texto Decifrado: SEGURANCA DA INFORMACAO
PS E:\01-Geral\PUC\9 Período\Segurança_d_inf\Entregas\criptografia-simetrica-cesar\s
rc>
```

Texto Cifrado : ZLNBYHUJH KH PUMVYTHJHV
Texto Decifrado: SEGURANCA DA INFORMACAO



Tarefa 2 — Decifração (Força Bruta)

Tarefa 2 — Decifre a mensagem

O texto abaixo foi cifrado com a Cifra de Cesar. Descubra a mensagem original e a chave utilizada:

FULSWRJUDILD

Dica: Use força bruta — tente cada valor de k de 1 a 25 e veja qual produz uma mensagem com sentido em português.

Decifrado



Decifrar a mensagem: `FULSWRJUDILD` (via força bruta)

- **Método:** testar sistematicamente as 25 chaves possíveis (deslocamentos de 1 a 25) até obter um texto legível.
- **Fórmula da decifração:** $P = (C - k + 26) \bmod 26$
- **Chave que gerou palavra válida:** $k = 3$

Demonstração ($k = 3$)

- $F (5) - 3 = 2 \rightarrow \mathbf{C}$
- $U (20) - 3 = 17 \rightarrow \mathbf{R}$
- $L (11) - 3 = 8 \rightarrow \mathbf{I}$
- $S (18) - 3 = 15 \rightarrow \mathbf{P}$
- $W (22) - 3 = 19 \rightarrow \mathbf{T}$
- $R (17) - 3 = 14 \rightarrow \mathbf{O}$
- $J (9) - 3 = 6 \rightarrow \mathbf{G}$
- $U (20) - 3 = 17 \rightarrow \mathbf{R}$
- $D (3) - 3 = 0 \rightarrow \mathbf{A}$
- $I (8) - 3 = 5 \rightarrow \mathbf{F}$
- $L (11) - 3 = 8 \rightarrow \mathbf{I}$
- $D (3) - 3 = 0 \rightarrow \mathbf{A}$

Resultado final

- Mensagem original: CRIPTOGRAFIA
- Chave utilizada: 3

Print

```
--- PROCESSANDO ---  
Texto Cifrado   : CRIPTOGRAFIA  
Texto Decifrado : FULSWRJUDILD
```

Texto Cifrado : CRIPTOGRAFIA
Texto Decifrado : FULSWRJUDILD



Implementação em Java



O algoritmo foi implementado em **Java**, com foco em performance e manutenção.

Melhorias aplicadas

1. Unificação da lógica em um único método `processarTexto()`, lidando com deslocamento positivo (cifração) e negativo (decifração).
2. Preservação de letras maiúsculas e minúsculas (*case-sensitive*), usando as bases ASCII (`'A'` e `'a'`).
3. Tratamento de chaves negativas antes do laço:
 - `deslocamento = chave % 26`
 - se `deslocamento < 0`, então `deslocamento += 26`

Validação

A execução confirmou os requisitos:

- receber as entradas (mensagem e k)

- gerar o criptograma
- restaurar o texto com o inverso aditivo da chave

```
=== LABORATÓRIO: CIFRA DE CÉSAR ===  
Digite o texto alvo: FUCK  
Informe o deslocamento (k): 7  
  
--- PROCESSAMENTO ---  
[+] Texto Cifrado    -> MBJR  
[-] Texto Restaurado -> FUCK
```

Codigo

```
import java.util.Scanner;  
  
public class CaesarCipher {  
  
    // Método único que serve tanto para cifrar quanto para  
    // decifrar  
    public static String processarTexto(String texto, int c  
have) {  
        StringBuilder textoFinal = new StringBuilder();  
  
        // Otimização: normaliza a chave antes do loop para  
        // sempre ser um deslocamento positivo (0-25).  
        // Isso evita fazer o "if" de número negativo a cad  
        // a iteração de caractere.  
        int deslocamento = chave % 26;  
        if (deslocamento < 0) {  
            deslocamento += 26;  
        }  
  
        // Percorre a string usando índice  
        for (int i = 0; i < texto.length(); i++) {  
            char c = texto.charAt(i);  
  
            // Aplica a regra matemática apenas para caract  
            // eres alfabéticos
```

```

        if (Character.isLetter(c)) {

            // Define a base ASCII dependendo se a letra
            // é maiúscula ou minúscula
            char baseLetra = Character.isUpperCase(c) ?
            'A' : 'a';

            // Aplica o deslocamento modular e converte
            // de volta para caractere
            char letraCifrada = (char) (baseLetra + (c
            - baseLetra + deslocamento) % 26);
            textoFinal.append(letraCifrada);
        } else {
            // Caracteres especiais, números e espaços
            // são mantidos intactos
            textoFinal.append(c);
        }
    }

    return textoFinal.toString();
}

public static void main(String[] args) {
    Scanner entrada = new Scanner(System.in);

    System.out.println("=== LABORATÓRIO: CIFRA DE CÉSAR
    ===");

    // 1. Receber mensagem e valor de k
    System.out.print("Digite o texto alvo: ");
    String textoBase = entrada.nextLine();

    System.out.print("Informe o deslocamento (k): ");
    int deslocamentoK = entrada.nextInt();

    System.out.println("\n--- PROCESSAMENTO ---");

    // 2. Cifrar e exibir

```

```

        String textoEncriptado = processarTexto(textoBase,
deslocamentoK);
        System.out.println("[+] Texto Cifrado    -> " + tex
toEncriptado);

        // 3. Decifrar e confirmar original (usando a mesma
função com chave negativa)
        String textoRestaurado = processarTexto(textoEncrip
tado, -deslocamentoK);
        System.out.println("[-] Texto Restaurado -> " + tex
toRestaurado);

        entrada.close();
    }
}

```



Tarefa 4 — Reflexão

1. Por que a Cifra de César é considerada insegura nos dias de hoje?

A Cifra de César é extremamente vulnerável porque o seu espaço de chaves é muito pequeno, contendo apenas 25 chaves possíveis (deslocamentos de 1 a 25). Um atacante consegue quebrar essa criptografia testando manualmente todas as combinações em questão de minutos por meio de um ataque de força bruta.

2. Quantas chaves precisariam ser testadas em um ataque de força bruta ao AES-128? Compare com as 25 chaves da Cifra de César.

No AES-128, existem 2^{128} chaves possíveis para serem testadas. Enquanto a Cifra de César possui apenas 25 combinações, o AES-128 possui um número tão massivo de possibilidades que levaria cerca de $5,4 \times 10^{18}$ anos para ser quebrado por um supercomputador, tornando-o, na prática, imune a ataques de força bruta com a tecnologia atual.

3. Como a Cifra de César se classifica: algoritmo de substituição ou de transposição? Justifique.

Classifica-se como um algoritmo de substituição. Isso ocorre porque cada elemento do texto em claro (cada letra) é mapeado e substituído por outro

elemento (a letra deslocada k posições no alfabeto). Ela não altera a ordem original das letras (o que caracterizaria uma transposição), apenas substitui os caracteres mantendo suas posições relativas na palavra.

4. Cite uma situação do cotidiano em que a criptografia simétrica (AES) é usada sem que percebamos.

O AES é o padrão mundial atual e está presente em ações cotidianas invisíveis aos usuários, como ao conectar-se a uma rede Wi-Fi (que utiliza protocolos WPA2 e WPA3), na navegação segura em sites (por meio do protocolo TLS no HTTPS) e no armazenamento seguro de dados em dispositivos.

ass: Murilo Bertelli